

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Abhilash J (1BM24CS006)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Abhilash J (1BM24CS006) , who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb2026 to June-2026 .The Lab report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	01
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	11
3.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	20
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	23
5.	Write a C program to simulate producer-consumer problem using semaphores	31
6.	Write a C program to simulate the concept of Dining Philosophers problem.	34
7.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	37

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1 : FCFS & SJF

Question : Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

A.Code : FCFS :

//program to simulate FCFS scheduling

#include <stdio.h>

```
void sort(int* at,int* bt,int* p,int n){    for(int i=0;i<n;i++){        for(int j=0;j<n-i-1;j++){
if(at[j] > at[j+1]){            int temp=at[j]; at[j] = at[j+1]; at[j+1] = temp;            temp =
bt[j]; bt[j] = bt[j+1]; bt[j+1] = temp;            temp = p[j]; p[j] = p[j+1] ; p[j+1] = temp;
        }
    }
}

} int main(){    int n;    printf("Enter the number of process:
");    scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],p[n];
    for(int i=0;i<n;i++){        printf("Enter the AT and BT of P%d : ",i+1);
p[i] = i+1;        scanf("%d %d",&at[i],&bt[i]);
    }    sort(at,bt,p,n);

    ct[0] = at[0] + bt[0];
    for(int i=1;i<n;i++){        if(ct[i-1] > at[i])
ct[i] = ct[i-1] + bt[i];        else            ct[i] = at[i] +
bt[i];
    }    int sum_tat = 0,sum_wt = 0;    printf("\n\nProcess
\tAT\tBT\tCT\tTAT\tWT\n");    for(int i=0;i<n;i++){        tat[i] = ct[i]-
```

```

at[i];    wt[i] = tat[i] - bt[i];    sum_tat += tat[i];    sum_wt +=
wt[i];    printf("p%d
\t\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\n\n");
    float avg_tat = (float)sum_tat/n,avg_wt = (float)sum_wt/n;    printf("Average TAT:
%.2f\nAverage WT: %.2f",avg_tat,avg_wt);    printf("\n\nPROGRAM DEVELOPED
Abhilash J");    return 0;
}

```

SJF :

```
// program to simulate SJF algorithm
#include <stdio.h>

void sort(int* at,int* bt,int* p,int n){    for(int i=0;i<n;i++){        for(int j=0;j<n-i-1;j++){
if(at[j] > at[j+1]){            int temp=at[j]; at[j] = at[j+1]; at[j+1] = temp;            temp =
bt[j]; bt[j] = bt[j+1]; bt[j+1] = temp;            temp = p[j]; p[j] = p[j+1] ; p[j+1] = temp;
        }
    }
}

} int main(){    int n;    printf("Enter the number of process:
");    scanf("%d",&n);

    int at[n],bt[n],ct[n],tat[n],wt[n],p[n],finished[n];

    for(int i=0;i<n;i++){
        printf("Enter the AT and BT of P%d : ",i+1);        p[i] = i+1;
        scanf("%d %d",&at[i],&bt[i]);        finished[i] = 0;
    }
    sort(at,bt,p,n);
    int completed=0,current_time = 0;
    while(completed < n){        int
idx=-1;        int min_bt=9999;
        for(int i=0;i<n;i++){            if(at[i]<=current_time && finished[i] == 0){
if(bt[i] < min_bt){                min_bt = bt[i];                idx = i;
            }
        }
    }

    if(idx != -1) {        ct[idx] = current_time + bt[idx];
tat[idx] = ct[idx] - at[idx];        wt[idx] = tat[idx] - bt[idx];
```

```

        current_time = ct[idx];        finished[idx] = 1;
completed++;
    }    else
current_time++;
    }
    int sum_tat = 0,sum_wt = 0;
    printf("\n\nProcess \tAT\tBT\tCT\tTAT\tWT\n");    for(int
i=0;i<n;i++){        sum_tat += tat[i];        sum_wt += wt[i];
printf("p%d
\t\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }    printf("\n\n");
    float avg_tat = (float)sum_tat/n,avg_wt = (float)sum_wt/n;    printf("Average TAT:
%.2f\nAverage WT: %.2f",avg_tat,avg_wt);    printf("\n\nPROGRAM DEVELOPED BY
ABHILASH J");
}

```

SRTF :

```

#include <stdio.h>
#include <limits.h>
int main() {
int n;
printf("Enter number of processes: ");    scanf("%d", &n);
    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], p[n];    for (int i = 0; i < n; i++) {
printf("Enter AT and BT for P%d: ", i + 1);        scanf("%d %d", &at[i], &bt[i]);
p[i] = i + 1;
    }
}

```

```

    for (int i = 0; i < n; i++) {        rt[i] = bt[i];
    }    int current_time = 0, completed = 0;
    while (completed < n) {        int idx =
-1;        int min_rt = INT_MAX;
        for (int i = 0; i < n; i++) {            if (at[i] <= current_time && rt[i] >
0) {                if (rt[i] < min_rt) {                    min_rt = rt[i];                    idx
= i;
                }
            else if (rt[i] == min_rt) {                if (at[i] < at[idx])
{                    idx = i;
                }
            }
        }
        if (idx == -1) {
current_time++;        } else {
rt[idx]--;        current_time++;
        if (rt[idx] == 0) {            completed++;
ct[idx] = current_time;            tat[idx] = ct[idx] - at[idx];
wt[idx] = tat[idx] - bt[idx];
        }
    }
    }
    float total_tat = 0, total_wt = 0;
    printf("\n\nProcess \tAT\tBT\tCT\tTAT\tWT\n");    for (int i = 0; i <
n; i++) {        total_tat += tat[i];        total_wt += wt[i];
        printf("P%d \t\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }    printf("\n-----");    printf("\nTotal Turnaround
Time: %.2f", total_tat);    printf("\nTotal Waiting Time:  %.2f", total_wt);

```

```

printf("\nAverage Turnaround Time: %.2f", total_tat / n);   printf("\nAverage Waiting
Time:  %.2f", total_wt / n);   printf("\n-----");
printf("\n\nPROGRAM DEVELOPED BY ABHILASH J\n");

return 0;
}

```

B.Output :

FCFS

```

PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhilash J\Documents\1BM24CS006\program
s.exe"
Enter the number of processes: 4
Enter the AT and BT of P1 : 0 2
Enter the AT and BT of P2 : 1 2
Enter the AT and BT of P3 : 5 3
Enter the AT and BT of P4 : 6 4

Process      AT      BT      CT      TAT      WT
p1           0       2       2       2       0
p2           1       2       4       3       1
p3           5       3       8       3       0
p4           6       4      12       6       2

Average TAT: 3.50
Average WT: 0.75

PROGRAM DEVELOPED BY Abhilash J

```

```

● PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhilash J\Documents\1BM24CS006\sjf.exe"
Enter the number of processes: 4
Enter AT and BT of P1: 0 6
Enter AT and BT of P2: 0 8
Enter AT and BT of P3: 0 7
Enter AT and BT of P4: 0 3

Process AT      BT      CT      TAT      WT
P1      0       6       9       9       3
P2      0       8      24      24      16
P3      0       7      16      16       9
P4      0       3       3       3       0

Average TAT = 13.00
Average WT  = 7.00

PROGRAM DEVELOPED BY ABHILASH J

```

SJF:

SRTF (SJF preemptive)

```

● PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhilash J\Documents\1BM24CS006\srtf.exe"
Enter number of processes: 5
Enter AT and BT for P1: 2 1
Enter AT and BT for P2: 1 5
Enter AT and BT for P3: 4 1
Enter AT and BT for P4: 0 6
Enter AT and BT for P5: 2 3

Process AT      BT      CT      TAT      WT
P1      2       1       3       1       0
P2      1       5      16      15      10
P3      4       1       5       1       0
P4      0       6      11      11       5
P5      2       3       7       5       2

-----
Total Turnaround Time = 33.00
Total Waiting Time    = 17.00
Average TAT           = 6.60
Average WT            = 3.40
-----

PROGRAM DEVELOPED BY ABHILASH J

```

PROGRAM 2 : PRIORITY (PREEMPTIVE&NON-PREEMPTIVE) ROUND ROBIN (diff time quantum)

Question : Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

A. Priority preemptive :

```
// Program to simulate the preemptive scheduling algorithms
// assuming LOWER THE NUMBER -> HIGHER THE PRIORITY
#include <stdio.h>
int main(){    int n;    printf("Enter the number of process : ");
scanf("%d",&n);
    int arr[n],at[n],bt[n],ct[n],wt[n],rt[n],pr[n],p[n],tat[n];

    for(int i=0;i<n;i++){        printf("Enter the AT,BT and priority of P%d : ",i+1);
p[i] = i+1;
        scanf("%d %d %d",&at[i],&bt[i],&pr[i]);        rt[i] = bt[i];
    }

    int completed = 0,current_time = 0;
    while(completed < n){        int idx
= -1;        int highest = 9999;
        for(int i=0;i<n;i++){            if(at[i] <= current_time && rt[i] > 0 && pr[i] <
highest){
                highest = pr[i];            idx = i;
            }
        }
        current_time += bt[idx];
        completed++;
        rt[idx] -= bt[idx];
        tat[idx] = current_time - at[idx];
        wt[idx] = rt[idx];
    }
    for(int i=0;i<n;i++){
        printf("Process %d: AT=%d, BT=%d, CT=%d, WT=%d, RT=%d, PR=%d, P=%d, TAT=%d\n",i+1,at[i],bt[i],ct[i],wt[i],rt[i],pr[i],p[i],tat[i]);
    }
}
```



```

    for(int i=0;i<n;i++){
printf("Enter the AT,BT and
priority of P%d : ",i+1);    p[i] =
i+1;    scanf("%d %d
%d",&at[i],&bt[i],&pr[i]);
finished[i] = 0;
    }    int completed = 0,current_time = 0;
    while(completed < n){    int idx
= -1;    int highest = 9999;
        for(int i=0;i<n;i++){    if(at[i] <= current_time && finished[i] == 0 && pr[i]
< highest){    highest = pr[i];
idx = i;
        }
    }
    if(idx != -1){    ct[idx] = current_time+bt[idx];
tat[idx] = ct[idx] - at[idx];    wt[idx] = tat[idx] - bt[idx];
current_time = ct[idx];
        finished[idx] = 1; // marking the process as finished    completed++;
    }    else
        completed++;
    }

    float total_tat = 0, total_wt = 0;
    printf("\n\nProcess \tAT\tBT\tPRIORITY\tCT\tTAT\tWT\n");    for (int i = 0; i <
n; i++) {    total_tat += tat[i];    total_wt += wt[i];    printf("P%d
\t\t%d\t%d\t%d\t\t%d\t%d\t\t%d\n", p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }    printf("\nAverage TAT: %.2f", total_tat / n);    printf("\nAverage
WT:  %.2f", total_wt / n);
    printf("\n\nPROGRAM DEVELOPED BY ABHILASH J\n");    return 0;
}

```

C.Round Robin :

```
#include <stdio.h>
int main() {    int
n, tq;

    printf("Enter the number of processes: ");    scanf("%d", &n);
printf("Enter Time Quantum: ");    scanf("%d", &tq);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], p[n];    int visited[n];

    for (int i = 0; i < n; i++) {        p[i] = i + 1;        printf("Enter AT
and BT of P%d: ", p[i]);        scanf("%d %d", &at[i], &bt[i]);
rt[i] = bt[i];        visited[i] = 0;
    }    int queue[100]; // Simple queue array    int front =
0, rear = 0;    int completed = 0, current_time = 0;

    while(completed < n) {        for(int i = 0; i < n; i++) {            if(at[i] <=
current_time && visited[i] == 0) {                queue[rear++] = i;
visited[i] = 1;
            }
        }        if (front == rear) {
current_time++;            continue;
        }
        int i = queue[front++];

        if (rt[i] > tq) {            current_time += tq;
rt[i] -= tq;
        } else {
            current_time += rt[i];            rt[i] = 0;
ct[i] = current_time;            tat[i] = ct[i] - at[i];
wt[i] = tat[i] - bt[i];            completed++;
        }
    }
```

```

        for(int j = 0; j < n; j++) {            if(at[j] <= current_time && visited[j]
== 0) {                queue[rear++] = j;            visited[j] = 1;
        }
    }

    if (rt[i] > 0) {
queue[rear++] = i;
        } } float sum_tat = 0, sum_wt = 0; printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) { sum_tat += tat[i]; sum_wt += wt[i];
printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    } printf("\nAverage TAT: %.2f", sum_tat / n);
printf("\nAverage WT: %.2f\n", sum_wt / n);
    return 0;
}

```

D.Output :

PRIORITY PREEMPTIVE :

```

PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhila
s.exe"
Enter the number of processes: 3
Enter AT, BT and Priority of P1: 0 5 3
Enter AT, BT and Priority of P2: 1 2 1
Enter AT, BT and Priority of P3: 2 4 2

Process AT      BT      Priority      CT      TAT      WT
P1      0      5      3      11      11      6
P2      1      2      1      3      2      0
P3      2      4      2      7      5      1

Average TAT: 6.00
Average WT : 2.33

```

PRIORITY NON-PREEMPTIVE :

```
PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhilash J\Documents\1BM24CS006\PriorityNonPreemptive.exe"
Enter the number of processes: 5
Enter AT, BT and Priority of P1: 0 3 5
Enter AT, BT and Priority of P2: 2 2 3
Enter AT, BT and Priority of P3: 3 5 2
Enter AT, BT and Priority of P4: 4 4 4
Enter AT, BT and Priority of P5: 6 1 1
```

Process	AT	BT	Priority	CT	TAT	WT
P1	0	3	5	3	3	0
P2	2	2	3	11	9	7
P3	3	5	2	8	5	0
P4	4	4	4	15	11	7
P5	6	1	1	9	3	2

Average TAT: 6.20
Average WT : 3.20

ROUND ROBIN :

```
PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhilash J\Documents\1BM24CS006\RoundRobin.exe"
Enter the number of processes: 5
Enter Time Quantum: 2
Enter AT and BT of P1: 0 5
Enter AT and BT of P2: 1 3
Enter AT and BT of P3: 2 1
Enter AT and BT of P4: 3 2
Enter AT and BT of P5: 4 3
```

PID	AT	BT	CT	TAT	WT
P1	0	5	13	13	8
P2	1	3	12	11	8
P3	2	1	5	3	2
P4	3	2	9	6	4
P5	4	3	14	10	7

Average TAT: 8.60
Average WT : 5.80

```

PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\User
Enter the number of processes: 5
Enter Time Quantum: 3
Enter AT and BT of P1: 0 5
Enter AT and BT of P2: 1 3
Enter AT and BT of P3: 2
1
Enter AT and BT of P4: 3 2
Enter AT and BT of P5: 4 3

PID    AT    BT    CT    TAT    WT
P1      0     5    11    11     6
P2      1     3     6     5     2
P3      2     1     7     5     4
P4      3     2     9     6     4
P5      4     3    14    10     7

Average TAT: 7.40
Average WT : 4.60

```

PROGRAM 3 : MULTILEVEL QUEUE

Question : Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

=> A.Code :

```
#include <stdio.h>
#include <string.h>

struct Process {
    int id;
    int arrival_time;
    int burst_time;
    char type[10];
    int completion_time;
    int turnaround_time;
    int waiting_time;
    int executed;
};

int main() {
    int n, i, current_time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);
```

```

struct Process p[n];

for (i = 0; i < n; i++) {
    p[i].id = i + 1;
    printf("\nProcess %d:\n", p[i].id);
    printf("Arrival Time: ");
    scanf("%d", &p[i].arrival_time);
    printf("Burst Time: ");
    scanf("%d", &p[i].burst_time);
    printf("Type (System/User): ");
    scanf("%s", p[i].type);
    p[i].executed = 0;
}

printf("\nExecution Order:\n");

while (completed < n) {
    int found = 0;

    for (i = 0; i < n; i++) {
        if (strcmp(p[i].type, "System") == 0 && p[i].executed == 0 && p[i].arrival_time <=
current_time) {
            current_time += p[i].burst_time;
            p[i].completion_time = current_time;
            p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
            p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
            p[i].executed = 1;
            completed++;
            printf("P%d (System) executed at time %d\n", p[i].id, current_time);
            found = 1;
        }
    }
}

```

```

        break;
    }
}

if (!found) {
    for (i = 0; i < n; i++) {
        if (strcmp(p[i].type, "User") == 0 && p[i].executed == 0 && p[i].arrival_time <=
current_time) {
            current_time += p[i].burst_time;
            p[i].completion_time = current_time;
            p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
            p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
            p[i].executed = 1;
            completed++;
            printf("P%d (User) executed at time %d\n", p[i].id, current_time);
            found = 1;
            break;
        }
    }
}

if (!found) {
    current_time++;
}
}

printf("\nID\tType\tArrival\tBurst\tWait\tTAT\n");
for (i = 0; i < n; i++) {
    total_wt += p[i].waiting_time;
    total_tat += p[i].turnaround_time;
}

```

```
        printf("%d\t%s\t%d\t%d\t%d\t%d\n", p[i].id, p[i].type, p[i].arrival_time, p[i].burst_time,
p[i].waiting_time, p[i].turnaround_time);
    }

    printf("\nAverage Waiting Time: %.2f", total_wt / n);
    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

    return 0;
}B.Output :
```

USN:1BM24CS006

Enter the number of processes: 4

Process 1:

Arrival Time: 0

Burst Time: 3

Type (System/User): System

Process 2:

Arrival Time: 1

Burst Time: 4

Type (System/User): User

Process 3:

Arrival Time: 5

Burst Time: 2

Type (System/User): System

Process 4:

Arrival Time: 6

Burst Time: 3

Type (System/User): User

Execution Order:

P1 (System) executed at time 3

P2 (User) executed at time 7

P3 (System) executed at time 9

P4 (User) executed at time 12

ID	Type	Arrival	Burst	Wait	TAT
1	System	0	3	0	3
2	User	1	4	2	6
3	System	5	2	2	4
4	User	6	3	3	6

Average Waiting Time: 1.75

Average Turnaround Time: 4.75

PROGRAM 4 : REAL TIME CPU SCHEDULING ALGORITHMS (RATE MONOTONIC,EARLIEST DEADLINE,PROPORTIONAL)

Question : Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

=>

A.Rate Monotonic :

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int find_lcm(int a, int b) {    int max = (a > b) ? a : b;    while (1) {        if  
(max % a == 0 && max % b == 0) return max;        max++;  
    }
```

```
} int main() {    int n, i, j, current_time = 0, hyperperiod = 1;    float  
utilization = 0;
```

```
    printf("Enter the number of processes: ");    scanf("%d", &n);
```

```
    int burst[n], period[n], remaining_burst[n];
```

```
    printf("Enter the CPU burst times:\n");    for (i = 0; i < n;  
i++) {        scanf("%d", &burst[i]);        remaining_burst[i]  
= burst[i];
```

```
    }    printf("Enter the time periods:\n");    for (i = 0; i < n; i++) {        scanf("%d",  
&period[i]);        if (i == 0) hyperperiod = period[i];        else hyperperiod =  
find_lcm(hyperperiod, period[i]);        utilization += (float)burst[i] / period[i];
```

```

    }    printf("LCM = %d\n", hyperperiod);    printf("\nRate Monotone
Scheduling:\n");    printf("PID\tBurst\tPeriod\n");    for (i = 0; i < n; i++) {
printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
    }
    printf("\n%f <= %f => true\n\n", utilization, n * (pow(2,
1.0/n) - 1));
    printf("Scheduling occurs for %d ms\n", hyperperiod);
    int last_pid = -1;    while (current_time <
hyperperiod) {        int selected = -1;        int
min_period = 10000;
        for (i = 0; i < n; i++) {            if (current_time % period[i] ==
0) {                remaining_burst[i] = burst[i];
            }
        }        for (i = 0; i < n; i++) {            if
(remaining_burst[i] > 0) {                if (period[i] < min_period)
{                    min_period = period[i];                    selected = i;
                }
            }
        }        if (selected != last_pid) {            if (selected != -1)
printf("%dms onwards: Process %d running\n", current_time, selected + 1);
else
        printf("%dms onwards: CPU is idle\n", current_time);
last_pid = selected;
        }        if (selected != -1) remaining_burst[selected]--;
current_time++;
    }
    printf("\n\nProgram Developed by : 1bm24cs006");    return 0;
}

```

B.Earliest Deadline first :

```
#include <stdio.h>
```

```

int main() {    int n, i, j, time = 0, completed = 0, total_time;
printf("Enter number of processes: ");    scanf("%d", &n);
    int burst[n], period[n], deadline[n], remaining[n], next_deadline[n];

    for (i = 0; i < n; i++) {        printf("Enter Burst, Period, and Deadline for P%d: ", i +
1);
        scanf("%d %d %d", &burst[i], &period[i], &deadline[i]);        remaining[i] =
burst[i];        next_deadline[i] = deadline[i];
    }    printf("Enter total simulation time: ");    scanf("%d",
&total_time);

    printf("\nScheduling occurs for %d ms\n", total_time);
    int last_pid = -1;    while (time <
total_time) {        int selected = -1;        int
min_deadline = 10000;

        for (i = 0; i < n; i++) {            if (time > 0 && time % period[i] == 0)
{                remaining[i] = burst[i];                next_deadline[i] = time +
deadline[i];
            }
        }
        for (i = 0; i < n; i++) {            if (remaining[i] > 0) {                if
(next_deadline[i] < min_deadline) {                    min_deadline =
next_deadline[i];                    selected = i;
                }
            }
        }
        if (selected != last_pid) {            if (selected != -1)
printf("%dms: Task %d is running.\n", time, selected + 1);            else
printf("%dms: CPU is idle.\n", time);            last_pid = selected;
        }        if (selected != -1) remaining[selected]--;        time++;    }
    return 0;
}

```

C.Proportional Scheduling :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {    int
id;    int weight;    int
stride;    int pass;
} Process; void runStrideScheduling(Process processes[], int n, int totalSteps) {
    // Calculate stride: 10000 / weight
    // Smaller stride = larger share = runs more often!    for (int i = 0; i < n; i++) {
        processes[i].stride = 10000 / processes[i].weight;        processes[i].pass = 0;
    }    for (int i = 0; i < totalSteps; i++) {        // Find process with the
smallest pass value        int best = 0;        for (int j = 1; j < n; j++) {
            if (processes[j].pass < processes[best].pass) {                best = j;
            }
        }
        printf("Running Process P%d (Pass: %d)\n",                processes[best].id,
processes[best].pass);

        // Advance the pass        processes[best].pass += processes[best].stride;
    }
}
int main() {
    // Define processes: ID, Weight
    Process processes[] = {{1, 10}, {2, 20}, {3, 5}};    int n = sizeof(processes) /
sizeof(processes[0]);
    runStrideScheduling(processes, n, 10);

    return 0;
}
```

D.Output :

RMS :

```
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100

LCM (Hyperperiod) = 100

Rate Monotonic Scheduling
PID    Burst  Period
1      20    50
2      35    100

CPU Utilization = 0.750
Bound = 0.828

Scheduling occurs for 100 ms

0 ms onwards : Process P1 running
20 ms onwards : Process P2 running
50 ms onwards : Process P1 running
70 ms onwards : Process P2 running
75 ms onwards : CPU Idle

PROGRAM DEVELOPED BY ABHILASH J
```

Earliest Deadline first :

```
Enter number of processes: 3
Enter Burst, Period and Deadline for P1: 2 4 5
Enter Burst, Period and Deadline for P2: 3 7 10
Enter Burst, Period and Deadline for P3: 2 8 10
Enter total simulation time: 20
```

Scheduling occurs for 20 ms

```
0 ms : Task P1 is running
2 ms : Task P2 is running
4 ms : Task P1 is running
6 ms : Task P2 is running
7 ms : Task P3 is running
8 ms : Task P1 is running
10 ms : Task P2 is running
12 ms : Task P1 is running
14 ms : Task P3 is running
16 ms : Task P1 is running
18 ms : Task P2 is running
```

PROGRAM DEVELOPED BY ABHILASH J

Proportional Scheduling :

```
Enter number of processes: 3
Enter weight for P1: 10
Enter weight for P2: 20
Enter weight for P3: 5
Enter number of scheduling steps: 10
```

Execution Order:

```
Step 1 : Running Process P1 (Pass = 0)
Step 2 : Running Process P2 (Pass = 0)
Step 3 : Running Process P3 (Pass = 0)
Step 4 : Running Process P2 (Pass = 500)
Step 5 : Running Process P1 (Pass = 1000)
Step 6 : Running Process P2 (Pass = 1000)
Step 7 : Running Process P2 (Pass = 1500)
Step 8 : Running Process P1 (Pass = 2000)
Step 9 : Running Process P2 (Pass = 2000)
Step 10 : Running Process P3 (Pass = 2000)
```

PROGRAM DEVELOPED BY ABHILASH J

PROGRAM 5 : PRODUCER-CONSUMER PROBLEM SEMAPHORE EDITION

Question : Write a C program to simulate producer-consumer problem using semaphores

=>

A.Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE]; int in
= 0; int out = 0;
sem_t empty; sem_t full;
pthread_mutex_t mutex;

void *producer(void *param) {    int
item;    while (1) {        item = rand() %
100;        sem_wait(&empty);
pthread_mutex_lock(&mutex);

        buffer[in] = item;        printf("Producer produced: %d at %d\n", item, in);
in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
sem_post(&full);        sleep(1);
    }
}
```

```

} void *consumer(void *param) {    int item;
while (1) {        sem_wait(&full);
pthread_mutex_lock(&mutex);
    item = buffer[out];    printf("Consumer consumed: %d at %d\n", item, out);
out = (out + 1) % BUFFER_SIZE;
    pthread_mutex_unlock(&mutex);
sem_post(&empty);    sleep(2);
    }
} int main() {
    pthread_t prod_tid, cons_tid;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod_tid, NULL, producer, NULL);
    pthread_create(&cons_tid, NULL, consumer, NULL);
    pthread_join(prod_tid, NULL);
    pthread_join(cons_tid, NULL);

    return 0;
}

```

B.Output :

```
PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Use
s.exe"
Producer produced: 41 at index 0
Consumer consumed: 41 from index 0
Producer produced: 67 at index 1
Producer produced: 34 at index 2
Consumer consumed: 67 from index 1
Producer produced: 0 at index 3
Producer produced: 69 at index 4
Consumer consumed: 34 from index 2
Producer produced: 24 at index 0
Consumer consumed: 0 from index 3
Producer produced: 78 at index 1
Producer produced: 58 at index 2
Consumer consumed: 69 from index 4
Producer produced: 62 at index 3
Producer produced: 64 at index 4
Consumer consumed: 24 from index 0
Producer produced: 5 at index 0
Consumer consumed: 78 from index 1
Producer produced: 45 at index 1
Consumer consumed: 58 from index 2
Producer produced: 81 at index 2
Consumer consumed: 62 from index 3
Producer produced: 27 at index 3
```

PROGRAM 6 : DINING PHILOSOPHER'S PROBLEM

Question : Write a C program to simulate the concept of Dining Philosophers problem.

=>

A.Code :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5      sem_t
fork_mutex[N];

void *philosopher(void *num) {    int i =
*(int *)num;
    while (1) {    printf("Philosopher %d is thinking.\n", i);
sleep(1);
        printf("Philosopher %d is hungry.\n", i);

        sem_wait(&fork_mutex[i]);    sem_wait(&fork_mutex[(i + 1) % N]);
        printf("Philosopher %d is eating.\n", i);    sleep(2);

        sem_post(&fork_mutex[i]);
sem_post(&fork_mutex[(i + 1) % N]);
        printf("Philosopher %d finished eating.\n", i);
    }
}

int main() {    pthread_t
thread[N];    int phil_id[N];
```

```
    for (int i = 0; i < N; i++)
sem_init(&fork_mutex[i], 0, 1);
    for (int i = 0; i < N; i++) {    phil_id[i] = i;
pthread_create(&thread[i], NULL, philosopher, &phil_id[i]);
    }    for (int i = 0; i < N; i++)
pthread_join(thread[i], NULL);

    return 0;
}
```

B.Output :

```
PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Users\Abhilash J\Documents\1BM24CS006\Programs\Philosophers.exe"
Philosopher 1 is thinking.
Philosopher 0 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is hungry.
Philosopher 0 is eating.
Philosopher 1 is hungry.
Philosopher 4 is hungry.
Philosopher 2 is hungry.
Philosopher 3 is hungry.
Philosopher 2 is eating.
Philosopher 3 is eating.
Philosopher 2 finished eating.
Philosopher 2 is thinking.
Philosopher 0 finished eating.
Philosopher 0 is thinking.
Philosopher 1 is eating.
Philosopher 0 is hungry.
Philosopher 2 is hungry.
Philosopher 4 is eating.
Philosopher 3 finished eating.
Philosopher 3 is thinking.
Philosopher 1 finished eating.
Philosopher 1 is thinking.
Philosopher 2 is eating.
Philosopher 1 is hungry.
Philosopher 3 is hungry.
Philosopher 1 is eating.
Philosopher 2 finished eating.
Philosopher 2 is thinking.
Philosopher 4 finished eating.
Philosopher 4 is thinking.
Philosopher 3 is eating.
```

PROGRAM 7 : BANKERS ALGORITHM

Question : Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

=>

A.Code :

```
#include <stdio.h>

int main() {    int n, m, i, j, k;    printf("Enter number of
processes: ");    scanf("%d", &n);    printf("Enter number
of resources: ");    scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m];    int f[n], ans[n],
ind = 0;

    printf("Enter Allocation Matrix:\n");    for (i = 0; i < n; i++)        for (j = 0; j <
m; j++) scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");    for (i = 0; i < n; i++)        for (j = 0; j < m;
j++) scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");    for (j = 0; j < m; j++)
scanf("%d", &avail[j]);    int need[n][m];    for (i = 0; i < n; i++)
for (j = 0; j < m; j++)        need[i][j] = max[i][j] - alloc[i][j];

    for (k = 0; k < n; k++) f[k] = 0;

    for (k = 0; k < n; k++) {        for (i = 0; i < n; i++) {            if
(f[i] == 0) {                int flag = 0;                for (j = 0; j < m; j++) {
if (need[i][j] > avail[j]) {                    flag = 1; break;
                }            }                if (flag == 0)
{                    ans[ind++] = i;
                }            }
        }
```

```

        for (j = 0; j < m; j++) avail[j] += alloc[i][j];        f[i] = 1;
    }
}
} }

printf("Following is the SAFE Sequence:\n");    for (i = 0; i < n - 1; i++)
printf(" P%d ->", ans[i]);    printf(" P%d\n", ans[n - 1]);

return 0;
}

```

B.Output :

```

PS C:\Users\Abhilash J\Documents\1BM24CS006> & "C:\Use
s.exe"
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

```